

DUY TAN UNIVERSITY

Da Nang

DS 423 SA – Machine Learning with Large Datasets

Face Mask Detection

Group 11 – Object Detection

Member	Student ID
Trần Văn Trường	28211452515
Nguyễn Như Nguyên	28211403542

July 10, 2026

Contents

1	Introduction and Problem Statement	2
2	Dataset	2
2.1	Source	2
2.2	Description	3
2.3	Class Distribution	3
2.4	Data Quality Issues Found and Corrected	3
3	Methodology	4
3.1	Pipeline Overview	4
3.2	Model Selection	4
3.3	Data Conversion	5
3.4	Training Configuration	5
3.5	Hyperparameter Tuning	5
4	Implementation	6
4.1	Code Availability	6
4.2	Converting Annotations to YOLO Format	6
4.3	Model Initialization and Training	7
4.4	Evaluation	7
5	Results and Evaluation	8
5.1	Training Curves	8
5.2	Overall Metrics	8
5.3	Per-Class Metrics	8
5.4	Qualitative Results	10
6	Individual Contribution	10
7	Conclusion and Future Work	11

Abstract

Public-health monitoring systems increasingly rely on automated visual inspection to verify mask compliance in crowded public spaces. This report presents an object-detection pipeline that identifies human faces in images and classifies each detected face as *with mask*, *without mask*, or *mask worn incorrectly*, together with the corresponding bounding box. The pipeline covers data collection, cleaning, exploratory data analysis, and feature engineering on the public Face Mask Detection dataset (853 images, three classes), followed by transfer learning with a YOLOv8s detector pretrained on COCO. The model was tuned and evaluated on a consumer-grade laptop GPU (NVIDIA RTX 3050, 4GB VRAM), demonstrating that a practical, resource-constrained deployment is feasible. Model performance is reported using mean Average Precision (mAP) at multiple IoU thresholds, Intersection-over-Union (IoU)-based matching, Precision, and Recall.

1 Introduction and Problem Statement

Manual verification of face-mask compliance in public spaces (transport hubs, hospitals, shopping centers) is labor-intensive and does not scale to the volume of people passing through such spaces every day. An automated computer-vision system that can process a camera feed and flag individuals who are not wearing a mask correctly would materially reduce the workload of public-health staff and provide continuous, objective monitoring.

This project frames mask-compliance checking as an **object detection** problem rather than a simple image-classification problem: a real image may contain zero, one, or several faces, each of which must be independently *located* (bounding box) and *classified* into one of three categories:

- `with_mask` – the person is wearing a mask correctly.
- `without_mask` – the person is not wearing a mask.
- `mask_worn_incorrect` – a mask is present but worn incorrectly (e.g., not covering the nose).

The goal of this project is to build, train, and evaluate an object detector that solves this task accurately enough to be useful in practice, while remaining trainable on a single consumer laptop GPU.

2 Dataset

2.1 Source

The project uses the publicly available **Face Mask Detection** dataset (andrewmvd, 2020) hosted on Kaggle: <https://www.kaggle.com/datasets/andrewmvd/face-mask-detection>.

2.2 Description

The dataset contains **853 images** annotated in Pascal-VOC XML format, with a total of approximately **4,072 bounding boxes** across three classes. Each XML annotation file specifies the image dimensions and, for every face in the image, a class label and a bounding box (x_{min} , y_{min} , x_{max} , y_{max}).

2.3 Class Distribution

Figure 1 shows a class imbalance typical of real-world safety datasets: the majority of annotated faces belong to the `with_mask` class (3,232 boxes, 79.4%), while `without_mask` (717 boxes, 17.6%) and especially `mask_worn_incorrect` (123 boxes, 3.0%) are minority classes. This imbalance was taken into account when splitting the data (stratified train/validation split) and when interpreting per-class Precision/Recall results (Section 3).

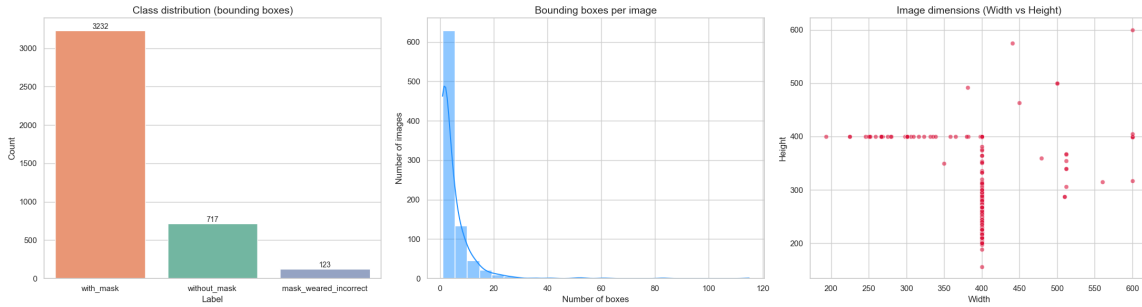


Figure 1: Exploratory Data Analysis: class distribution, number of bounding boxes per image, and image-size distribution.

2.4 Data Quality Issues Found and Corrected

During data cleaning, 12 bounding boxes (out of 4,072) were found to extend one or more pixels beyond the image boundary recorded in the corresponding XML annotation (i.e., $x_{min}/y_{min} < 0$ or x_{max}/y_{max} greater than the width/height fields parsed from the XML metadata). Rather than discarding these boxes outright, their coordinates were **clamped to the image boundary as recorded in the XML metadata** ($x_{min}, y_{min} \rightarrow \max(\text{value}, 0)$; $x_{max}, y_{max} \rightarrow \min(\text{value}, \text{width/height})$), and would only have been dropped if they became degenerate (zero or negative width/height) after clamping – in practice, no boxes were degenerate, so all 4,072 boxes were retained. This preserved all of the already-limited training data while eliminating a bug that would otherwise cause the annotation pipeline to raise coordinate-range errors during training.

3 Methodology

3.1 Pipeline Overview

The full pipeline is organized into ten parts inside a single Jupyter notebook (`group11_facemask.ipynb`), split according to each member's assigned role:

1. Data collection and loading (parsing Pascal-VOC XML annotations).
2. Data cleaning and preprocessing (missing values, out-of-bounds boxes, label encoding, verifying image files exist).
3. Exploratory Data Analysis (class balance, boxes-per-image, image-size distribution, annotated sample images).
4. Feature engineering (box width/height/area, aspect ratio, area ratio relative to image size).
5. Stratified train/validation split (80/20).
6. Model selection and building (dataset conversion to YOLO format; YOLOv8s initialization from COCO-pretrained weights).
7. Training setup, hyperparameter tuning, and full training run.
8. Evaluation (mAP, IoU-based Precision/Recall).
9. Prediction visualization on validation samples.
10. Saving the final model and a results summary table.

3.2 Model Selection

YOLOv8s (Ultralytics) was selected as the detection model for the following reasons:

- As a single-stage detector, YOLOv8 offers substantially faster training and inference than two-stage detectors such as Faster R-CNN, while still achieving competitive accuracy – an important property for a system intended to eventually run on continuous video/camera streams.
- The "small" (s) variant balances accuracy and computational cost, fitting comfortably within the 4GB VRAM budget of the training GPU (NVIDIA RTX 3050 Laptop), unlike the larger m/l/x variants.
- Ultralytics YOLOv8 ships with COCO-pretrained weights, enabling transfer learning, which is essential given the relatively small size of the Face Mask Detection dataset (853 images).
- The `ultralytics` library provides built-in data augmentation, automatic mixed-precision (AMP) training, and native computation of mAP/Precision/Recall, reducing the amount of custom training code needed and the surface area for bugs.

3.3 Data Conversion

Because YOLO expects images and label files in its own format, the cleaned annotations were converted from Pascal-VOC XML to YOLO text format: each line `<class_id> <x_center> <y_center> <width> <height>`, with all four coordinates normalized to $[0, 1]$ relative to the image size, and `class_id` converted from the 1-indexed label used earlier (`with_mask=1`, `without_mask=2`, `mask_wearred_incorrect=3`) to YOLO’s required 0-indexed scheme.

3.4 Training Configuration

Table 1: Training configuration used for the final YOLOv8s model.

Setting	Value
Base weights	yolov8s.pt (COCO-pretrained)
Image size	416×416
Batch size	8
Epochs	60 (early stopping, patience = 15)
Optimizer	Ultralytics auto optimizer (resolved to AdamW, lr=0.001429; see Section 3.5 note)
Initial learning rate (lr0, as passed to <code>train()</code>)	0.005 (selected via search; overridden by auto optimizer)
Mixed precision (AMP)	Enabled
DataLoader workers	0 (avoids multiprocessing issues on Windows)
Hardware	NVIDIA GeForce RTX 3050 Laptop GPU (4GB VRAM)

3.5 Hyperparameter Tuning

A small hyperparameter search was performed over the initial learning rate ($lr0 \in \{0.01, 0.005, 0.001\}$), training each candidate for a short number of epochs and comparing validation mAP@0.50. The best-performing learning rate, 0.005, was then passed as `lr0` to the full 60-epoch training run.

Note on the effective learning rate. The `ultralytics train()` call used `optimizer="auto"` (the library default, not overridden in this project). With this setting, Ultralytics automatically selects its own optimizer and learning rate based on the model and dataset size, and *ignores* any `lr0` value supplied by the caller – the training log explicitly records `"optimizer=auto" found, ignoring 'lr0=0.005'"` and reports the optimizer actually used as `AdamW(lr=0.001429, momentum=0.9)`. In other words, the value selected by the hyperparameter search (`lr0=0.005`) was not the learning rate actually applied during the

final 60-epoch run. This does not invalidate the search itself (it still correctly identified the best-performing candidate among the three tested within a fixed 3-epoch budget), but the effective optimizer/learning rate for the reported final model was chosen automatically by Ultralytics rather than by this search. A corrected version of this experiment would either set an explicit optimizer (e.g., SGD or AdamW) so that the searched `lr0` takes effect, or would run the search directly on Ultralytics' auto-selected settings. This search was intentionally kept small in scope, given the time and compute constraints of training on a single personal laptop.

4 Implementation

This section highlights the key parts of the implementation. The full, runnable, and commented source code is provided in `group11_facemask.ipynb`.

4.1 Code Availability

The complete, runnable, and commented notebook (`group11_facemask.ipynb`), together with a short README describing how to reproduce the results, is hosted at:

- **GitHub repository:** <https://github.com/NhuNguyen2307/Face-Mask-Detection.git>
- **Google Drive (trained weights / large files, if applicable):** <https://drive.google.com/drive/folders/1d2VqVXUiYbnAjqdw4f5thnWs3q0C1v-?usp=sharing>

4.2 Converting Annotations to YOLO Format

```
1 def convert_split_to_yolo(data_list, images_out_dir, labels_out_dir):
2     """Copy images + generate YOLO-format .txt annotation files
3     for one split (train/val)."""
4     n_boxes = 0
5     for item in data_list:
6         filename = item['filename']
7         src_path = item['image_path']
8         dst_path = os.path.join(images_out_dir, filename)
9         if not os.path.exists(dst_path):
10             shutil.copyfile(src_path, dst_path)
11
12         img_w, img_h = item['width'], item['height']
13         lines = []
14         for (x1, y1, x2, y2), label_id in zip(item['boxes'], item['labels']
15             ]):
16             class_id = int(label_id) - 1 # YOLO expects 0-based ids
17             x_center = min(max(((x1 + x2) / 2.0) / img_w, 0.0), 1.0)
18             y_center = min(max(((y1 + y2) / 2.0) / img_h, 0.0), 1.0)
```

```

18         box_w = min(max((x2 - x1) / img_w, 0.0), 1.0)
19         box_h = min(max((y2 - y1) / img_h, 0.0), 1.0)
20         lines.append(f"{class_id} {x_center:.6f} {y_center:.6f} "
21                     f"{box_w:.6f} {box_h:.6f}")
22         n_boxes += 1
23
24         label_path = os.path.join(
25             labels_out_dir, os.path.splitext(filename)[0] + ".txt")
26         with open(label_path, "w") as f:
27             f.write("\n".join(lines))
28     return n_boxes

```

4.3 Model Initialization and Training

```

1 from ultralytics import YOLO
2
3 model = YOLO("yolov8s.pt") # COCO-pretrained weights
4
5 train_run = model.train(
6     data=DATA_YAML_PATH,
7     imgsz=416,
8     batch=8,
9     workers=0, # avoid DataLoader issues on Windows
10    device=device_str, # '0' for GPU, 'cpu' otherwise
11    seed=SEED,
12    amp=True, # mixed precision -> lower VRAM usage
13    epochs=60,
14    lr0=0.005, # chosen via the hyperparameter search
15    patience=15, # early stopping
16    project=OUTPUT_DIR,
17    name="yolov8s_facemask_final",
18    plots=True,
19 )

```

4.4 Evaluation

```

1 map_results = model.val(
2     data=DATA_YAML_PATH, imgsz=416, batch=8,
3     device=device_str, workers=0, split="val"
4 )
5
6 print(f"mAP@[0.5:0.95] : {map_results.box.map:.4f}")
7 print(f"mAP@0.50      : {map_results.box.map50:.4f}")
8 print(f"mAP@0.75      : {map_results.box.map75:.4f}")
9 print(f"Precision (mp) : {map_results.box.mp:.4f}")

```



```
10 print(f"Recall (mr)      : {map_results.box.mr:.4f}")
```

In addition to the metrics computed natively by Ultralytics, a manual IoU-based matching routine (using `torchvision.ops.box_iou`) was implemented to compute per-class Precision and Recall at a fixed IoU threshold of 0.5, giving an explicit, independently verifiable view of the IoU metric required by the assignment, separate from the internally IoU-averaged mAP score.

5 Results and Evaluation

5.1 Training Curves

Figure 2 shows the training and validation loss (box-regression loss and classification loss) across epochs.

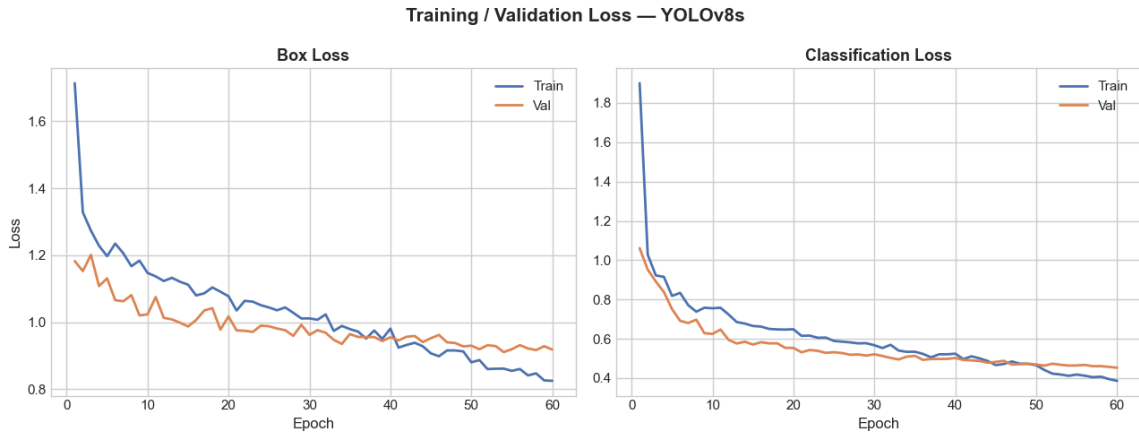


Figure 2: Training / validation loss curves for YOLOv8s.

5.2 Overall Metrics

Table 2: Overall detection performance on the validation split.

Metric	Value
mAP@[0.5:0.95]	0.5723
mAP@0.50	0.8186
mAP@0.75	0.6795
Precision (mean, IoU $\geq$ 0.5)	0.9293
Recall (mean, IoU $\geq$ 0.5)	0.7432

5.3 Per-Class Metrics

Table 3 reports the per-class metrics computed natively by Ultralytics (`model.val()`), and Table 4 reports the independent, manually-implemented Precision/Recall using

`torchvision.ops.box_iou` at a fixed IoU threshold of 0.5 and confidence threshold of 0.25 (Section 4.4). The two computations are in close agreement, cross-validating the evaluation pipeline.

Table 3: Per-class metrics from Ultralytics `model.val()` (171 validation images, 763 instances).

Class	Instances	Precision	Recall	mAP@0.50	mAP@[.5:.95]
All (mean)	763	0.929	0.743	0.819	0.572
with_mask	580	0.943	0.912	0.958	0.687
without_mask	156	0.902	0.705	0.800	0.523
mask_wearred_incorrect	27	0.943	0.612	0.698	0.508

Table 4: Manual IoU-based Precision/Recall ($\text{IoU} \geq 0.5$, confidence ≥ 0.25).

Class	TP	FP	FN	Precision	Recall
with_mask	540	65	40	0.8926	0.9310
without_mask	117	26	39	0.8182	0.7500
mask_wearred_incorrect	16	5	11	0.7619	0.5926

Figure 3 shows Average Precision, Precision, and Recall broken down by class, computed at an IoU threshold of 0.5.

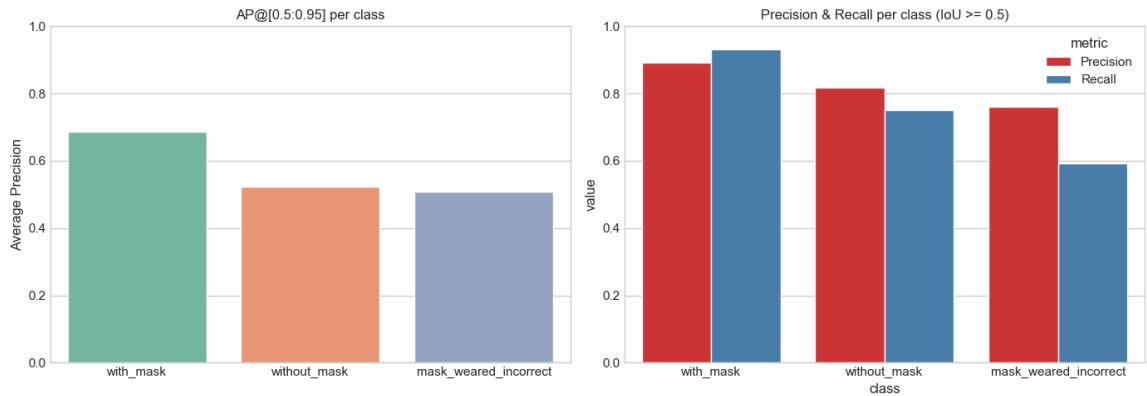


Figure 3: Per-class AP@[0.5:0.95] and Precision/Recall at $\text{IoU} \geq 0.5$.

The per-class chart shows that `with_mask` obtains the strongest AP and recall, while `mask_wearred_incorrect` has the lowest AP and recall among the three classes. This is consistent with the dataset imbalance discussed in Section 3: the minority incorrect-mask class provides fewer visual examples, making it harder for the detector to learn robust boundary and class cues.

5.4 Qualitative Results

Figure 4 shows example predictions on validation images, overlaid with predicted bounding boxes, class labels, and confidence scores.

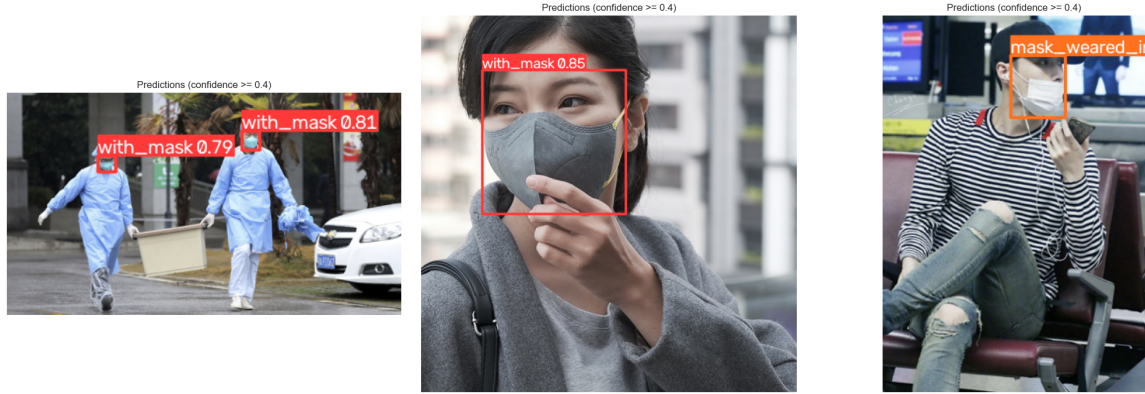


Figure 4: Sample model predictions on validation images.

6 Individual Contribution

Table 5: Individual contribution breakdown.

Member	Main Contributions	Share
Tran Van Truong	Data collection and loading; data cleaning and preprocessing; exploratory data analysis with charts; feature engineering; compiled the APA reference list.	50%
Nguyen Nhu Nguyen	Model selection and building; training and hyperparameter tuning; evaluation with the required metrics and result charts; built the PowerPoint presentation; wrote this LaTeX report and the conclusion.	50%

Both members jointly integrated and tested the final notebook end to end and reviewed each other's work before submission.

7 Conclusion and Future Work

This project demonstrated a complete, end-to-end object-detection pipeline for face-mask compliance monitoring, from raw XML annotations to a trained and evaluated YOLOv8s model, entirely reproducible on a single consumer laptop GPU. On the validation split, the final model achieved mAP@[0.5:0.95] of 0.5723, mAP@0.50 of 0.8186, and mAP@0.75 of 0.6795, with mean Precision of 0.9293 and mean Recall of 0.7432 at IoU ≥ 0.5 . These results indicate that YOLOv8s is practically useful for image-level face-mask compliance detection under controlled dataset conditions, although the lower minority-class recall shows that further data collection is still needed before real-world deployment.

Future work could include:

- Collecting additional `mask_worn_incorrect` examples to reduce class imbalance and improve minority-class recall.
- Training larger YOLOv8 variants (m/l) on a GPU with more VRAM to assess whether model capacity, rather than data quantity, is the limiting factor.
- Evaluating the model on real video/CCTV footage rather than static images, and measuring inference throughput (FPS) on the target deployment hardware.
- Exploring model export (e.g., ONNX, TensorRT) for faster on-device inference in a real monitoring system.

References

- [1] andrewmvd. (2020). *Face Mask Detection* [Data set]. Kaggle. <https://www.kaggle.com/datasets/andrewmvd/face-mask-detection>
- [2] Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. (2020). Albumentations: Fast and flexible image augmentations. *Information*, 11(2), 125. <https://doi.org/10.3390/info11020125>
- [3] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Version 8.1.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
- [4] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024–8035.
- [5] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779–788. <https://doi.org/10.1109/CVPR.2016.91>

- [6] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 91–99.